Multiple Traffic Light Controller with Cross Walk and an Emergency Buzzer with a SMART Sensor

CEIS114 Final Course Project

May 2024

Barbara Espericueta

Prof. **Alireza Kavianpour**

Introduction

Multiple Traffic Light Controller with Cross Walk and an Emergency Buzzer with a SMART Sensor:

- Overall Project: Creating a Multi-Intersection Traffic Light System using Wokwi Lab, exploring the intersection of electronics, microcontroller programming, and traffic engineering. Gaining insight into the principles behind traffic light signal control and practical skills that can contribute to more efficient and sustainable urban transportation systems.
- Objective: To design and develop a two-way traffic light controller with a pedestrian crossing and an emergency signal.
- *Tools: Wokwi Lab, Arduino IDE, IoT MQTT Panel, Microsoft Word, PowerPoint, and Wix.*

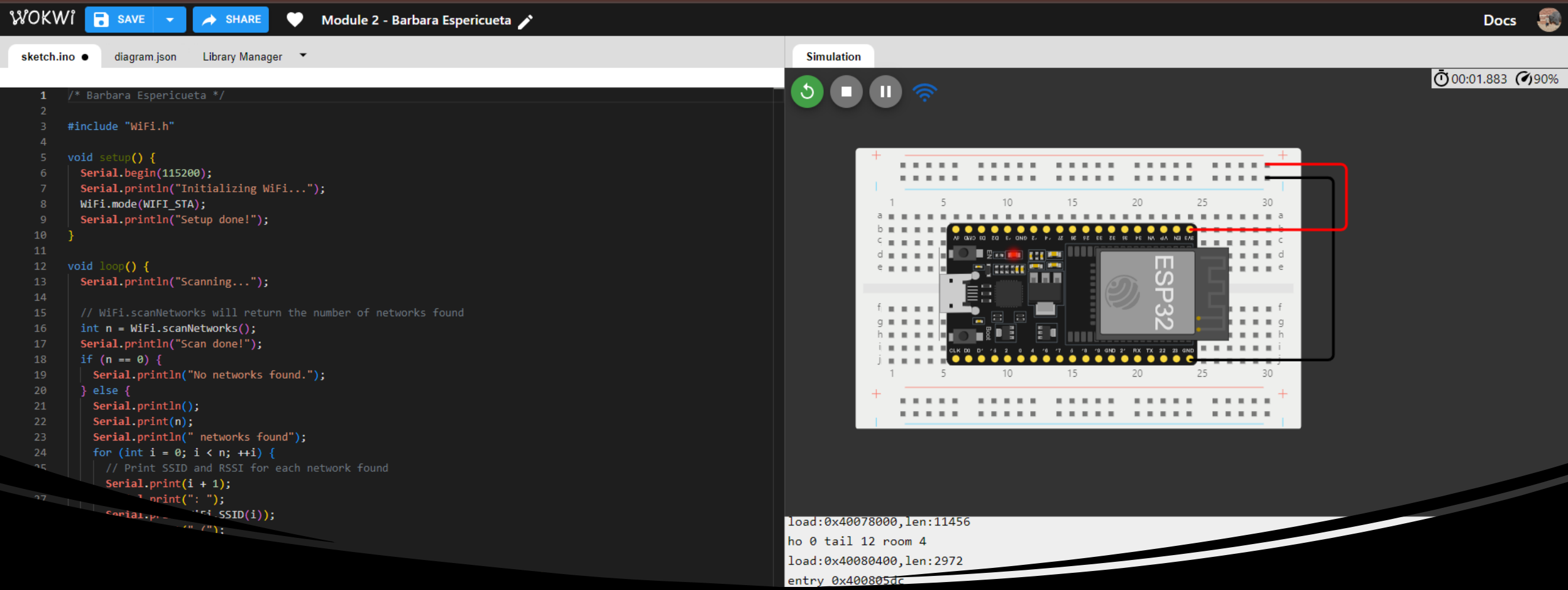
CEIS 114
Module 2

Project Plan for IoT Traffic Controller

In this module, we familiarized ourselves with the two-way traffic light controller and set up our ESP32 microcontroller board in the Wokwi virtual environment.

ESP32 (Screenshot)

Microcontroller mounted and powered ON



```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Initializing WiFi...
Setup done!
Scanning...
Scan done!

1 networks found
1: Wokwi-GUEST (-97)
```

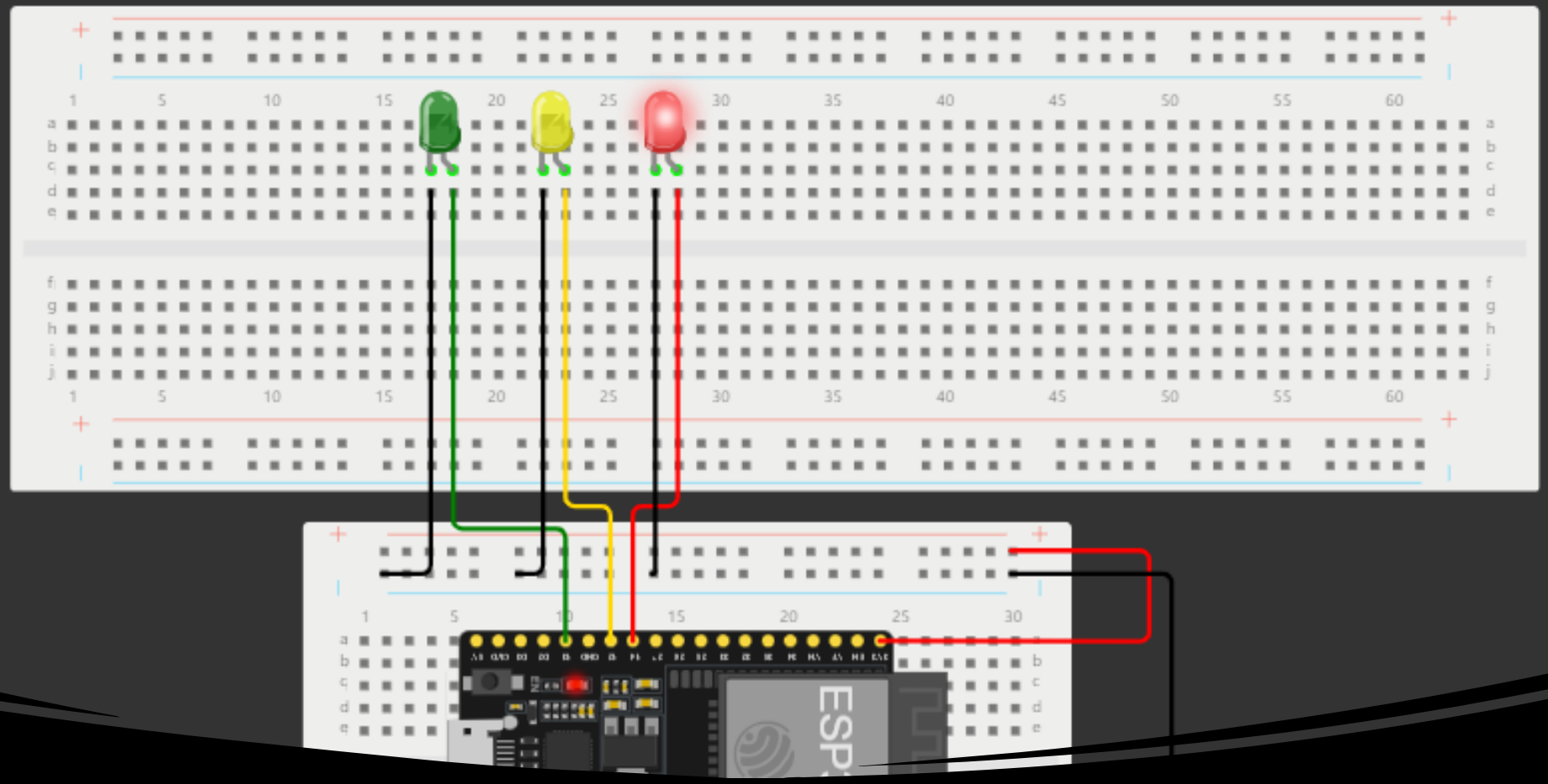
ESP32 Wi-Fi Scan

Screenshot of **Serial Monitor** showing the available networks

CEIS 114
Module 3

Creating the Traffic Controller

In this module, a program is executed to simulate a traffic light. In this step, we controlled a single LED light and then learned to control three LEDs to stimulate a single traffic light.



Picture of circuit with working LEDs

- ESP 32 Board
- Colored LEDs: Red, Yellow and Green
- 220 Ohm Resistors (optional)
- Wires
- Breadboard

Screenshot of code in Arduino IDE

WOKWI SAVE SHARE Module 3 - Barbara Espericueta

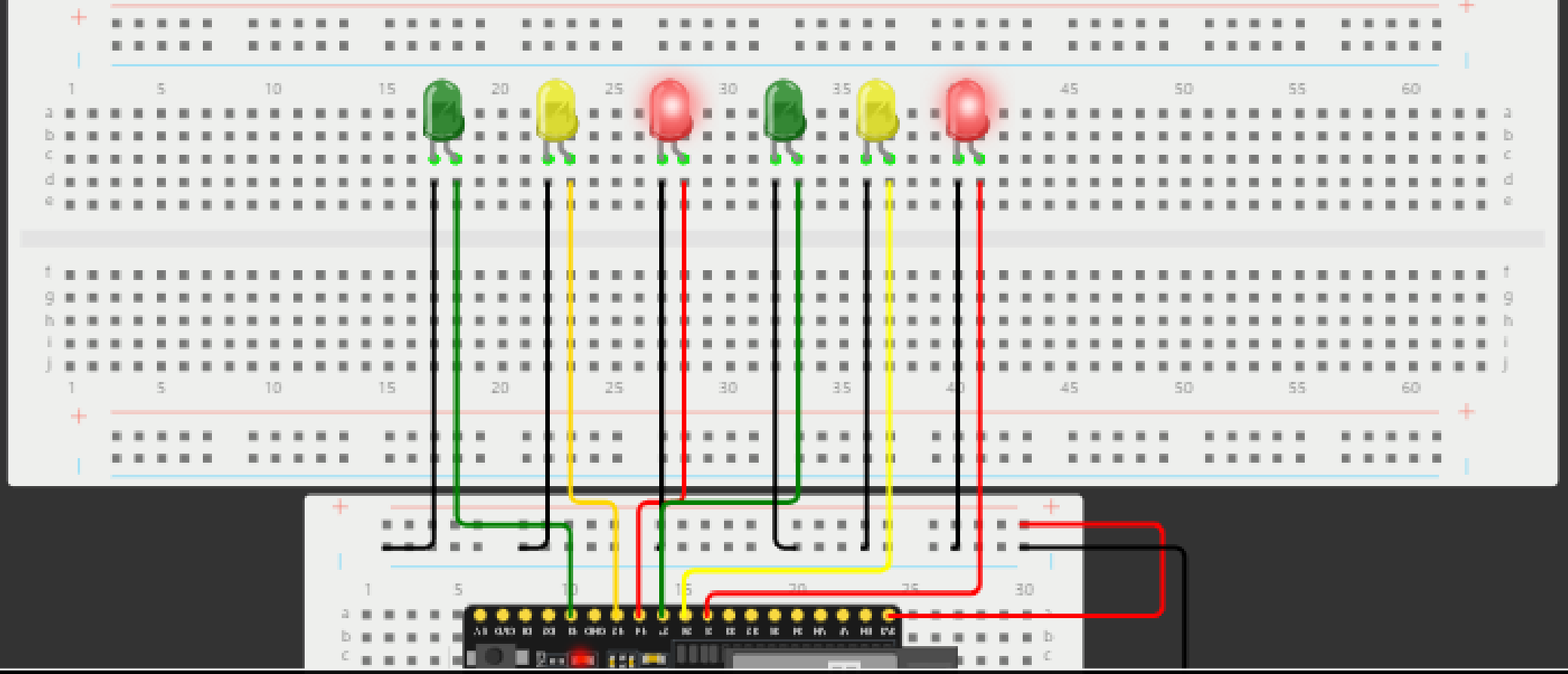
sketch.ino diagram.json Library Manager

```
// === Barbara Espericueta ====
// Module #3 project

const int red_LED1  = 14;       // The red LED1 is wired to ESP32 board pin GPIO14
const int yellow_LED1  = 12;     // The yellow LED1 is wired to ESP32 board pin GPIO12
const int green_LED1 = 13;  // The green LED1 is wired to ESP32 board pin GPIO13

// the setup function runs once when you press reset or power the board
void setup() {
 pinMode(red_LED1, OUTPUT);     // initialize digital pin GPIO14 (Red LED1) as an output.
pinMode(yellow_LED1, OUTPUT);   // initialize digital pin GPIO12 (yellow LED1) as an output.
pinMode(green_LED1, OUTPUT);      // initialize digital pin GPIO13 (green LED1) as an output.
}

// the loop function runs over and over again forever
void loop() {
  // The next three lines of code turn on the red LED1
  digitalWrite(red_LED1, HIGH);       // This should turn on the RED LED1
  digitalWrite(yellow_LED1 , LOW);    //  This should turn off the YELLOW LED1
  digitalWrite(green_LED1, LOW);      //  This should turn off the GREEN LED1

  delay(2000);                      // wait for 2 seconds

  // The next three lines of code turn on the green LED1
  digitalWrite(red_LED1, LOW);         // This should turn off the RED LED1
  digitalWrite(yellow_LED1 , LOW);        // This should turn off the YELLOW LED1
  digitalWrite(green_LED1, HIGH);        // This should turn on the GREEN LED1

  delay(2000);                    // wait for 2 seconds

// The next three lines of code turn on the yellow  LED1
   digitalWrite(red_LED1, LOW);       // This should turn off the RED LED1
  digitalWrite(yellow_LED1 , HIGH);   //  This should turn on the YELLOW LED1
  digitalWrite(green_LED1, LOW);        //  This should turn off the GREEN LED1

  delay(2000);                      // wait for 2 seconds
}
```

CEIS 114
Module 4

Creating a Multiple Traffic Light Controller

In this phase of the module, we implemented a program for simulating a multi-intersection traffic light controller. Encompassing a total of two traffic light systems.



Picture of circuit with working LEDs

- ESP 32 Board
- Colored LEDs: Red, Yellow and Green (two sets)
- Wires
- Breadboard

clk drv:0x00,q drv:0x00,d drv:0x00,cs0 drv:0x00,hd drv:0x00,wp drv:0x00

Screenshot of code in Wokwi

WOKWI SAVE SHARE Module 4 - Barbara Espericueta

sketch.ino ● diagram.json ● Library Manager

```
// === Barbara Espericueta ====
// Module #4 project

// Define some labels
const int red_LED1 = 14; // The red LED1 is wired to ESP32 board pin GPIO14
const int yellow_LED1 =12; // The yellow LED1 is wired to ESP32 board pin GPIO12
const int green_LED1 = 13; // The green LED1 is wired to ESP32 board pin GPIO13
const int red_LED2 = 25; // The red LED2 is wired to Mega board pin GPIO25
const int yellow_LED2 = 26; // The yellow LED2 is wired to Mega board pin GPIO 26
const int green_LED2 = 27; // The green LED2 is wired to Mega board pin GPIO 27

// the setup function runs once when you press reset or power the board
void setup() {
pinMode(red_LED1, OUTPUT); // initialize digital pin GPIO14 (Red LED1) as an output.
pinMode(yellow_LED1, OUTPUT); // initialize digital pin GPIO12 (yellow LED1) as an output.
pinMode(green_LED1, OUTPUT); // initialize digital pin GPIO13 (green LED1) as an output.
pinMode(red_LED2, OUTPUT); // initialize digital pin GPIO25(Red LED2) as an output.
pinMode(yellow_LED2, OUTPUT); // initialize digital pin GPIO26 (yellow LED2) as an output.
pinMode(green_LED2, OUTPUT); // initialize digital pin GPIO27 (green LED2) as an output.
}

// the loop function runs over and over again forever
void loop() {

// The next three lines of code turn on the red LED1
digitalWrite(red_LED1, HIGH); // This should turn on the RED LED1
digitalWrite(yellow_LED1 , LOW); // This should turn off the YELLOW LED1
digitalWrite(green_LED1, LOW); // This should turn off the GREEN LED1

delay(1000); //Extended time for Red light#1 before the Green of the other side turns ON

// The next three lines of code turn on the green LED2 for 2 seconds
digitalWrite(red_LED2, LOW); // This should turn off the RED LED2
digitalWrite(yellow_LED2 , LOW); // This should turn off the YELLOW LED2
digitalWrite(green_LED2, HIGH); // This should turn on the GREEN LED2

delay(2000); // wait for 2 seconds

```

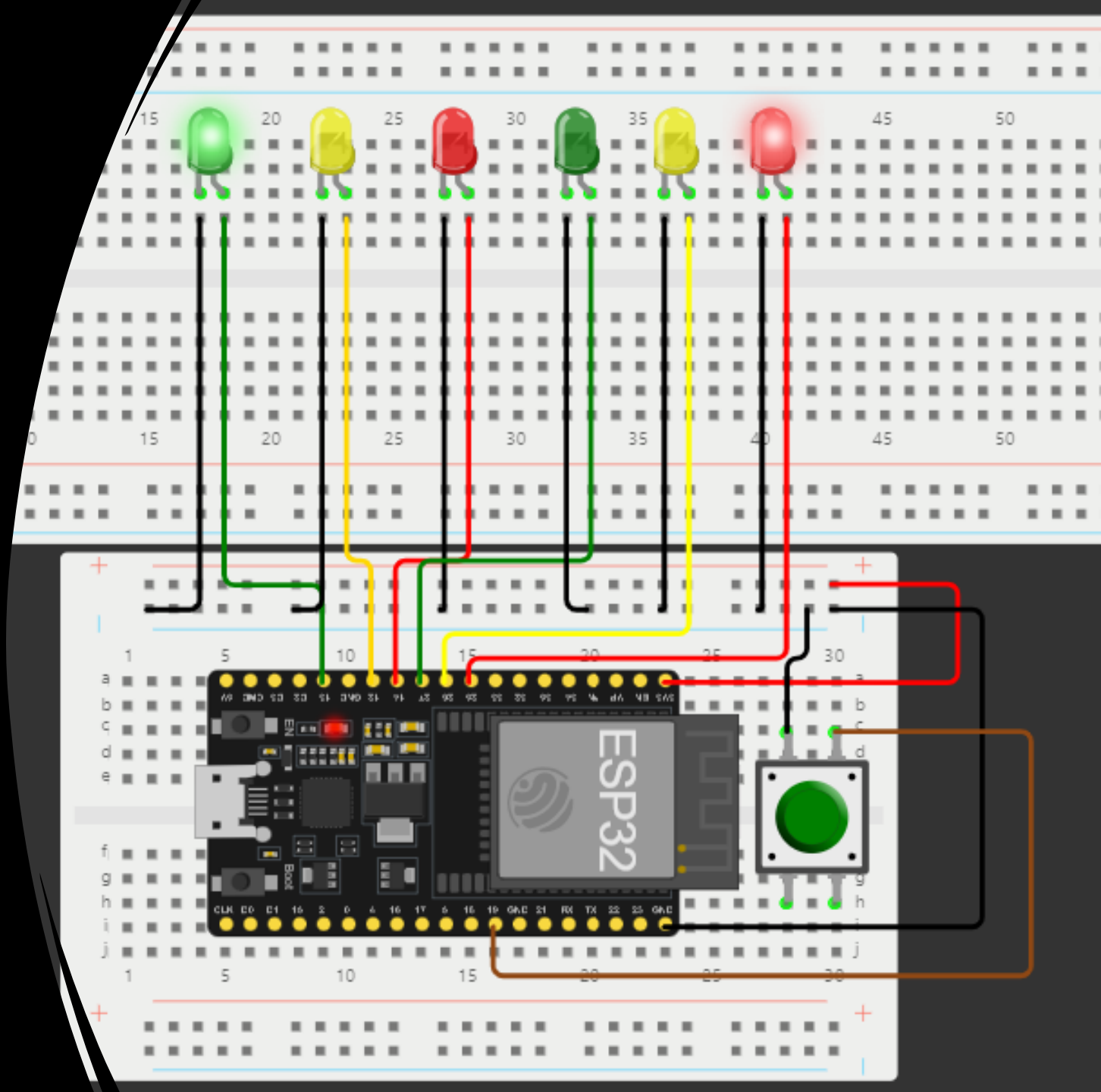
CEIS 114
Module 5

Creating a Multiple Traffic Light Controller with a Cross Walk

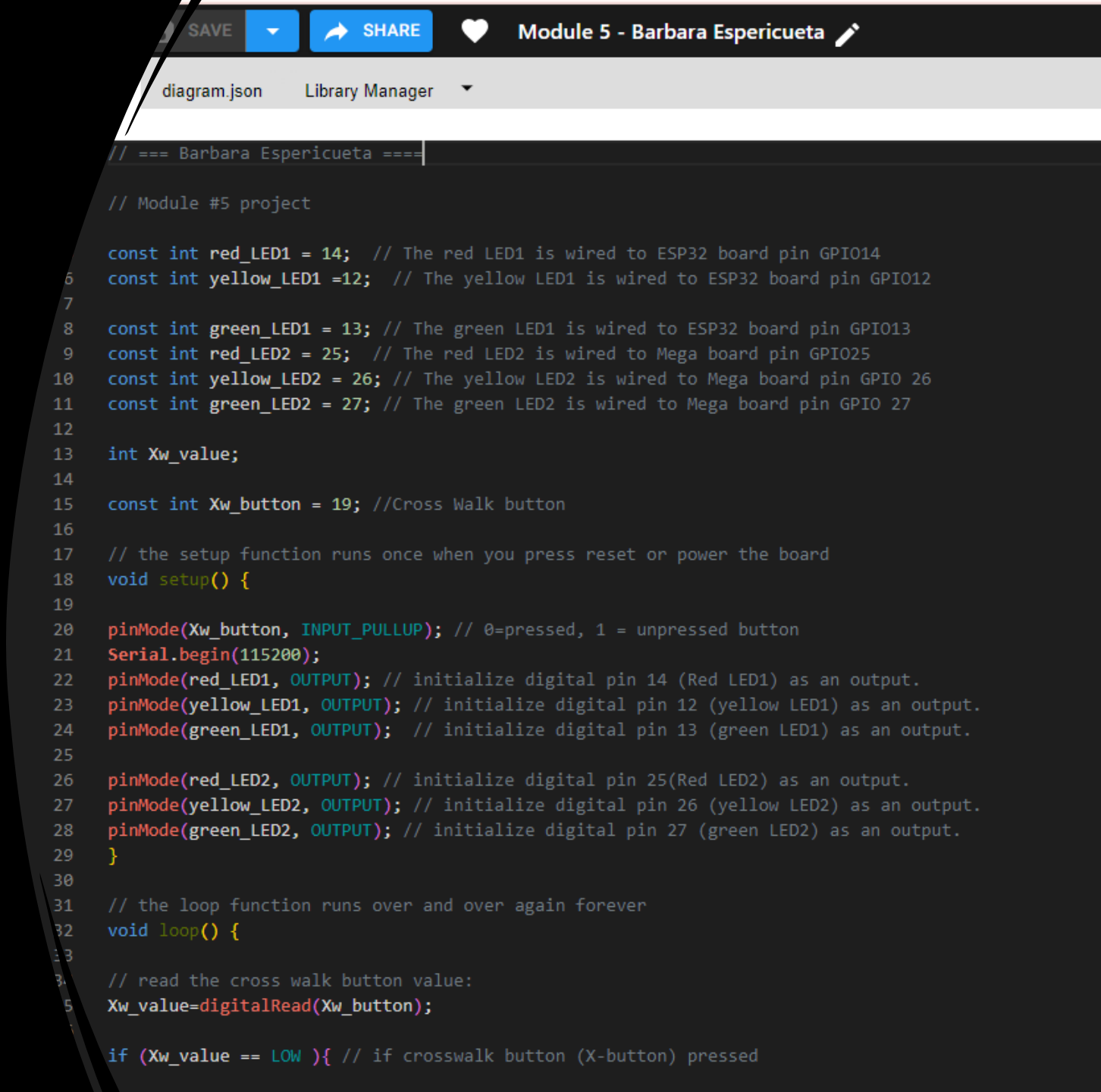
In this module, we embarked on the creation of a smart traffic light system with a crosswalk for people to walk safely. To make sure everyone stays safe and moves at the right time we created a sophisticated multiple traffic light controller integrated with a pedestrian crosswalk.

Screenshot of circuit with working LEDs

- ESP 32 Board
- Colored LEDs: Red, Yellow and Green (two sets)
- 220 Ohm Resistors (optional)
- Push Button
- Wires
- Breadboard

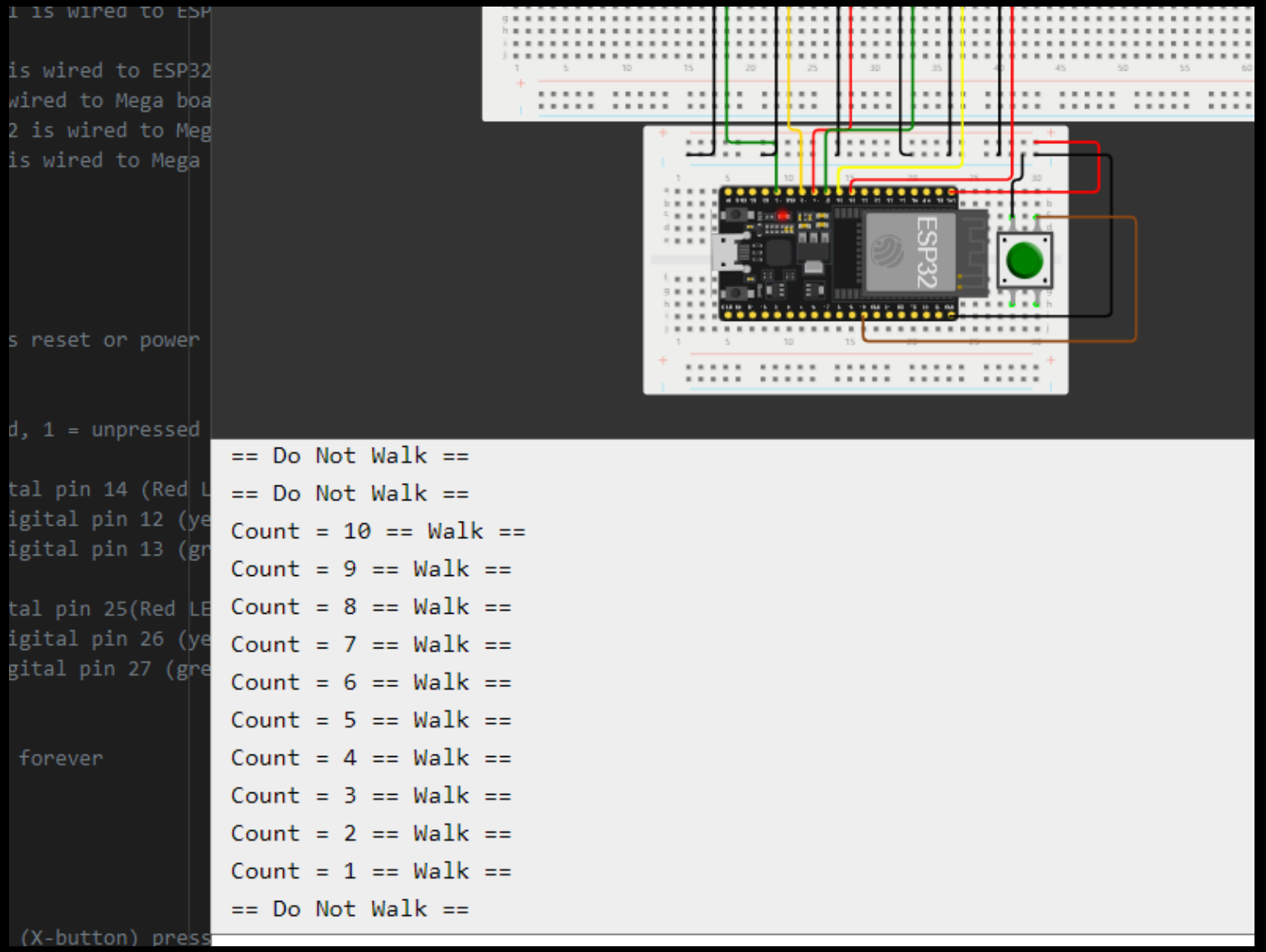


Screenshot of code in Wokwi



Screenshot of Serial Monitor in Wokwi

Screenshot of output in Serial Monitor



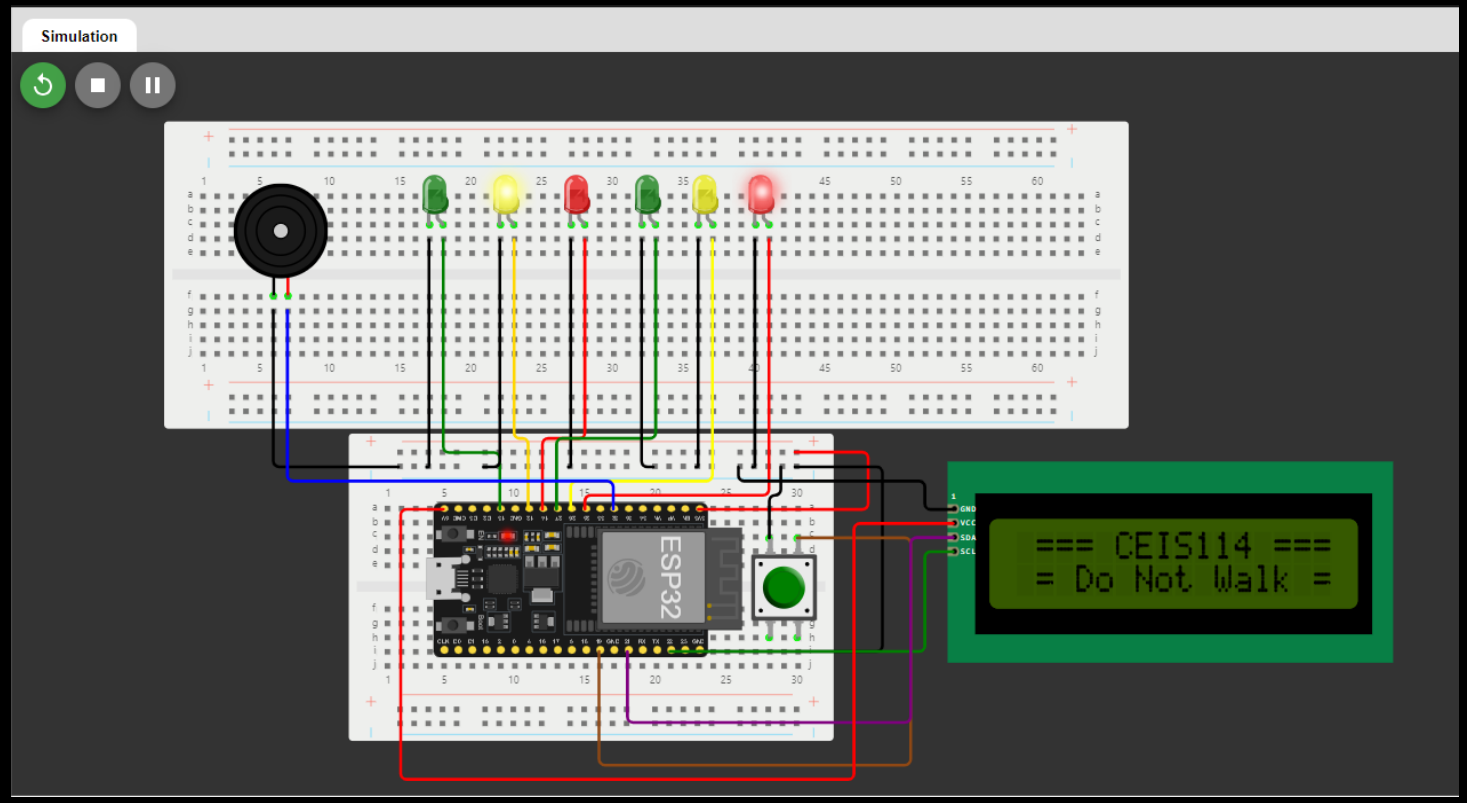
CEIS 114
Module 6

Creating a Multiple Traffic Light Controller with a Cross Walk and an Emergency Buzzer

In this phase of the module, we embarked on the creation of a smart traffic light system with a crosswalk of a display for people to see the instructions, and a buzzer to sound to alert them when to walk and not to walk at the intersection.

Picture of the circuit with working LEDs and LCD display

- ESP 32 Board
- Colored LEDs: Red, Yellow and Green (two sets)
- 220 Ohm Resistors (optional)
- Push Button
- LCD Unit with Message Display
- Wires
- Breadboard



Screenshot of code in Code Editor

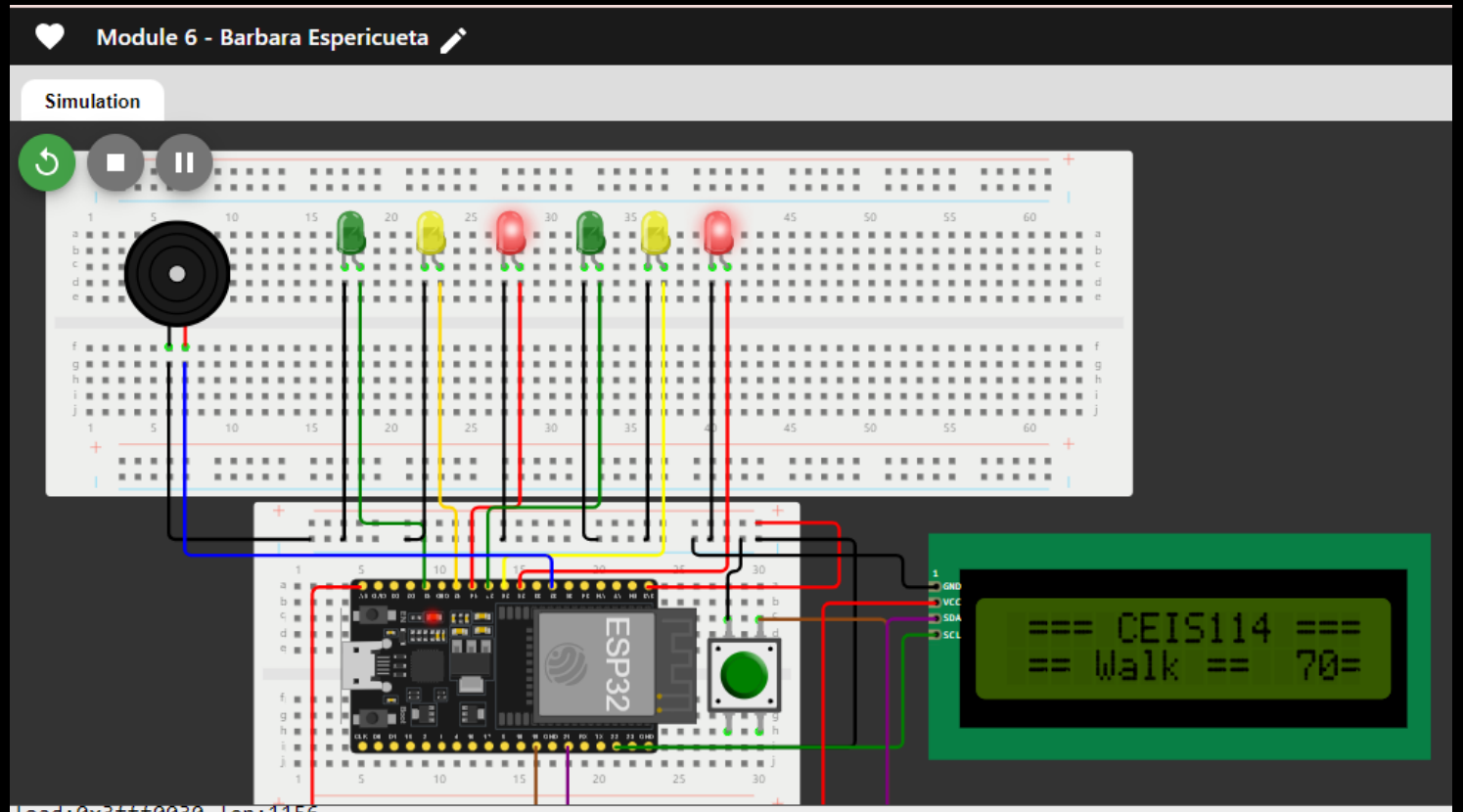
SAVE SHARE Module 6 - Barbara Espericueta

diagram.json libraries.txt Library Manager

```
// === Barbara Espericueta ====
// Module #6 project #include <Wire.h> //lcd
#include <LiquidCrystal_I2C.h> //lcd

LiquidCrystal_I2C lcd(0x27,16,2); //set the LCD address to 0x3F for a 16 chars and 2-li
// if it does not work then try 0x3F, if both addresses do not work then run the scan

const int bzr=32; // GPIO32 to connect the Buzzer
//=================== LCD ====================
const int red_LED1 = 14;  // The red LED1 is wired to ESP32 board pin GPIO14
const int yellow_LED1 =12;  // The yellow LED1 is wired to ESP32 board pin GPIO12
const int green_LED1 = 13; // The green LED1 is wired to ESP32 board pin GPIO13
const int red_LED2 = 25;  // The red LED2 is wired to Mega board pin GPIO25
const int yellow_LED2 = 26; // The yellow LED2 is wired to Mega board pin GPIO 26
const int green_LED2 = 27; // The green LED2 is wired to Mega board pin GPIO 27

int Xw_value;
const int Xw_button = 19; //Cross Walk button

void setup()
{

Serial.begin(115200);
pinMode(Xw_button, INPUT_PULLUP); // 0=pressed, 1 = unpressed button

lcd.init(); // initialize the lcd lcd.backlight();
lcd.setCursor(0,0); // column#4 and Row #1
lcd.print(" === CEIS114 ===");
pinMode(bzr,OUTPUT);

pinMode(red_LED1, OUTPUT); // initialize digital pin 14 (Red LED1) as an output.
pinMode(yellow_LED1, OUTPUT); // initialize digital pin12 (yellow LED1) as an output.
pinMode(green_LED1, OUTPUT);  // initialize digital pin 13 (green LED1) as an output.
```

Screenshot of Serial Monitor

Screenshot of output in Serial Monitor

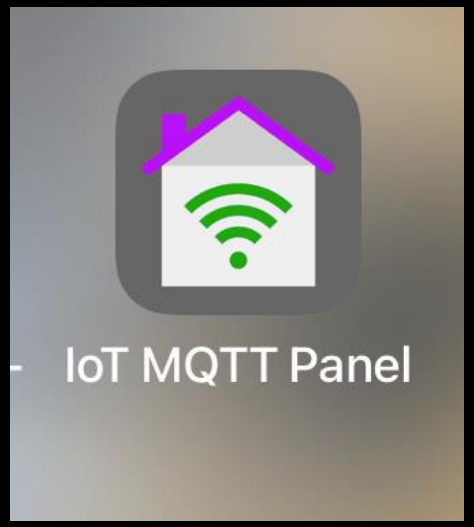


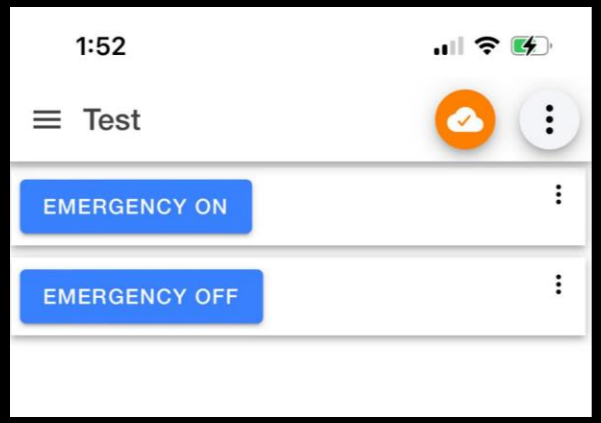

```
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
 == Do Not Walk ==
 Count = 10 == Walk ==
 Count = 9 == Walk ==
 Count = 8 == Walk ==
 Count = 7 == Walk ==
```

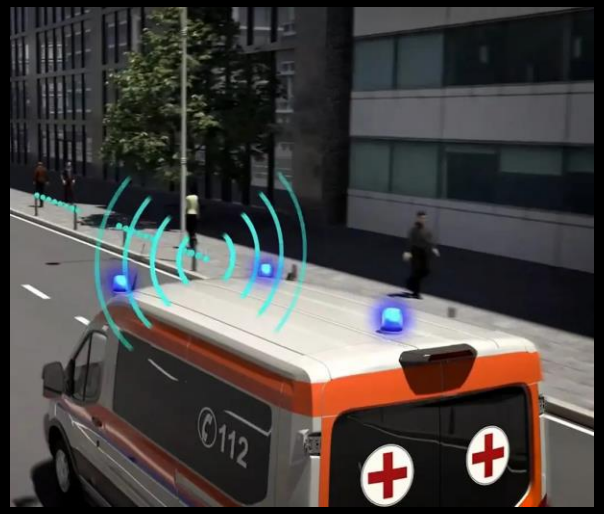
CEIS 114
Week 7 Project

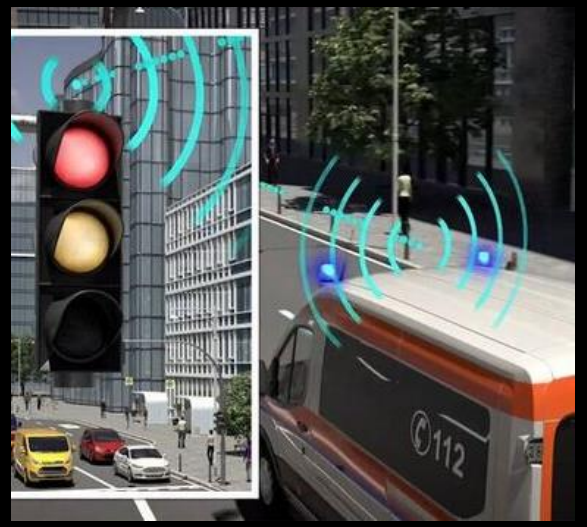
Creating a Multiple Traffic Light Controller with a Cross Walk and an Emergency Buzzer with secured IoT Control via Web

In this final step of our multi-intersection traffic light system, we entered the world of connected devices and created our own simple IoT system. We used the MQTT protocol, which is an easy-to-understand language that devices use to talk to each other.

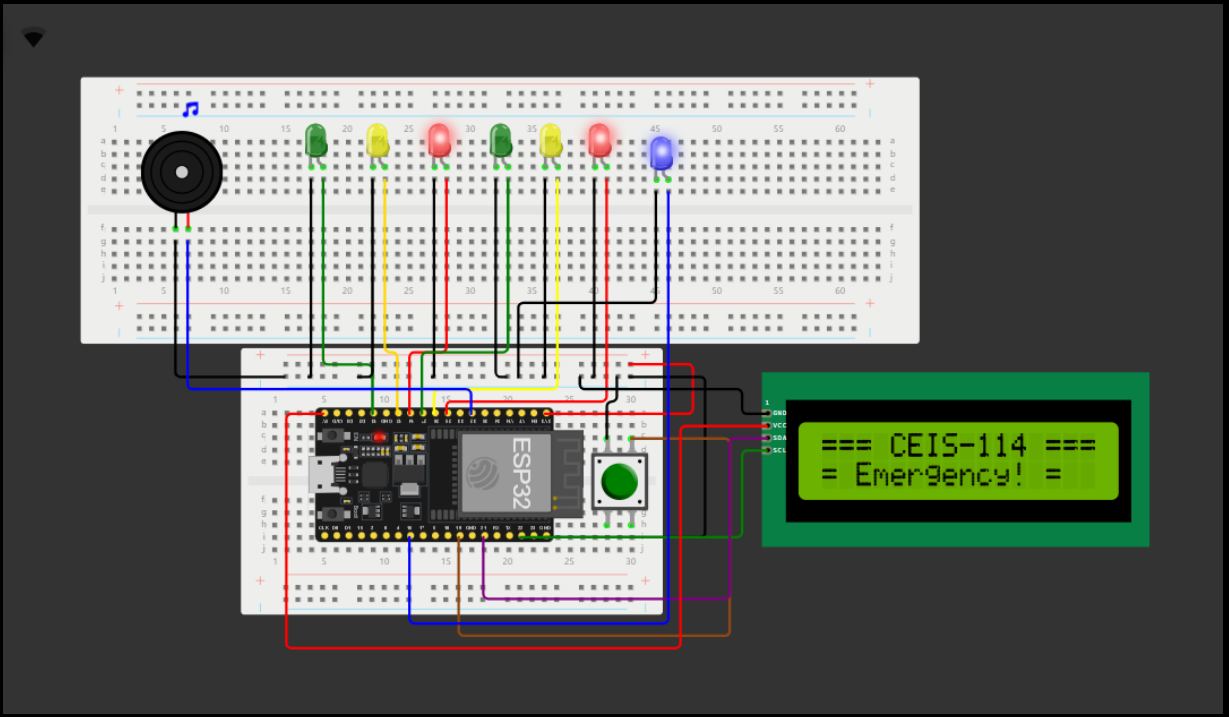






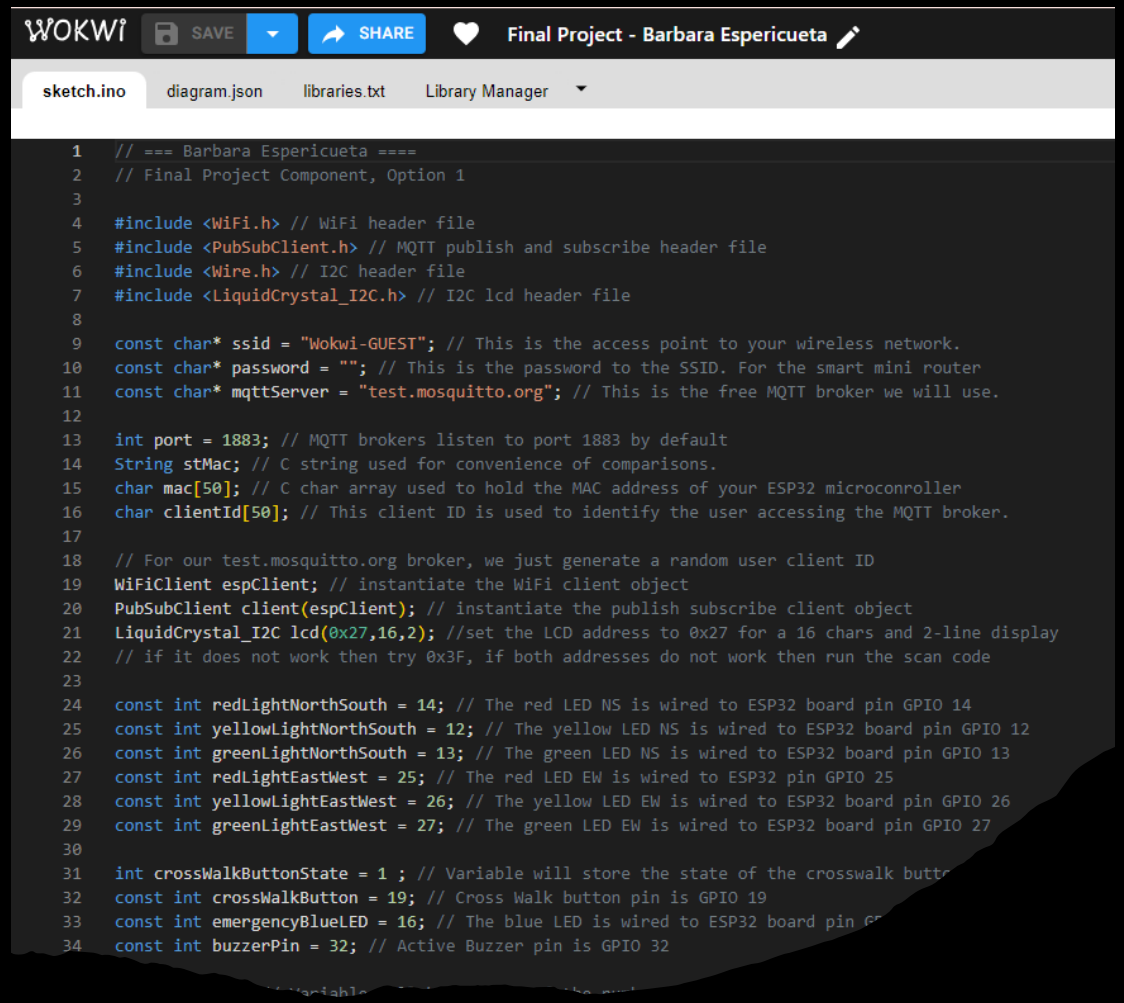


Screenshot of circuit **with working LEDs and LCD display** (Building/Operation)



- ESP 32 Board
- Colored LEDs: Red, Yellow and Green (two sets)
- One Blue LED – Emergency Light
- Push Button
- LCD Unit
- Buzzer
- Wires
- Breadboard

Screenshot of **code in Code Editor** (Testing)



```
11  char* mqttServer = "test
12
13  rt = 1883; // MQTT broke
14   stMac; // C string used
15  ac[50]; // C char array
16  lientId[50]; // This cli
17
18   our test.mosquitto.org
19  ient espClient; // insta
20  Client client(espClient)
21  Crystal_I2C lcd(0x27,16,
22  it does not work then tr
23
24  int redLightNorthSouth =
25  int yellowLightNorthSout
26  int greenLightNorthSouth
27  int redLightEastWest = 2
28  int yellowLightEastWest
29  int greenLightEastWest =
30
31  ossWalkButtonState = 1 ;
32  int crossWalkButton = 19
```

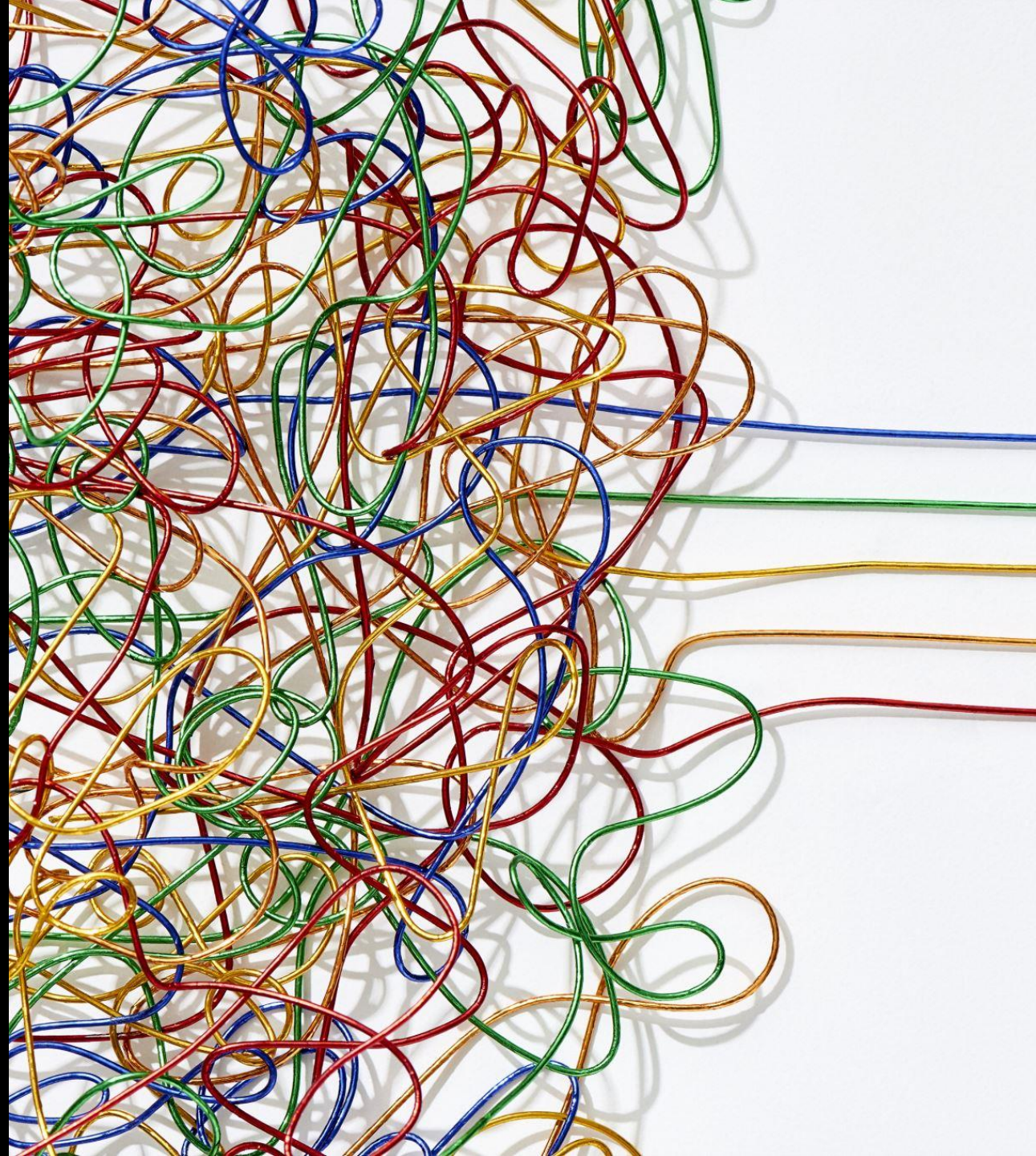
```
 == Do Not Walk ==
 == Do Not Walk ==
 == Do Not Walk ==
 == Walk == 15
 == Walk == 14
 == Walk == 13
 == Walk == 12
= Emergency! =
= Emergency! =
= Emergency! =
= Emergency! =
 == Do Not Walk ==
 == Do Not Walk ==
 == Do Not Walk ==
```

Screenshot of Serial Monitor (Testing)

Screenshot of output in Serial Monitor

Challenges/Lessons Learned

A challenge I faced during this project was the wiring between the display screen and the ESP32. After looking at a data sheet on the ESP32, I found that it was easier to understand what goes where, by referencing through GPI PIN Numbers. Then understood that 8 pins could be used to allow communication, 16 PWD for analog outputs, 2 digital-analog converters, 2 I2S interfaces, and 10 Capacitive sensing GPIOs. Then I was able to identify that the display screen had two connections for communication, one for voltage, and a ground. Lessons learned in this project, completing a project gives you more knowledge than just reading or watching.



Career Skills

Skills I developed that will benefit my career would be getting familiar with basic electronics: circuits, sensors, and programming microcontrollers. Programming is a skill I sharpened, gaining proficiency in the programming language in writing the software that runs the microcontroller a must-have skill in cybersecurity. Skills in Networking is another skill I gained. Wi-Fi, LAN, WAN, and cellular networks are essential, devices that can connect to the internet and communicate with other devices are growing fast knowing these skills will help any organization.

Conclusion

With the exponential growth in connected devices, each item in the IoT system communicates packets of data that require reliable connectivity, storage, and security. These projects will help us deal with such challenges as monitoring and securing large volumes of data.